

01_index-html

#html #css #javascript #frontend #api #SIEM #SOC

1 Ubicación del archivo dentro del proyecto

```
siem-lab/  
└─ frontend/  
    └─ index.html
```

El archivo `index.html` se encuentra en la raíz de la carpeta del frontend:

```
frontend/
```

Este archivo define la página principal de la interfaz web del laboratorio SIEM MVP.

Su función es mostrar el listado de alertas, permitir aplicar filtros básicos, gestionar la paginación y acceder al detalle de cada alerta.

2 Comando utilizado para visualizar el archivo

```
cd ~/siem-lab  
sed -n '1,260p' frontend/index.html
```

Desglose del comando:

```
cd ~/siem-lab
```

Sitúa la terminal en la raíz del proyecto.

```
sed
```

Ejecuta el programa `sed`, utilizado para visualizar contenido de archivos.

```
-n
```

Evita imprimir todo el archivo automáticamente.

```
'1,260p'
```

Indica que se impriman las líneas de la 1 a la 260.

```
frontend/index.html
```

Es la ruta del archivo que se quiere visualizar.

3 Código completo del archivo

```
<!doctype html>  
<html lang="es">  
<head>  
  <meta charset="utf-8" />  
  <meta name="viewport" content="width=device-width,initial-scale=1" />  
  <title>SIEM Lab – Alertas</title>  
  <link rel="stylesheet" href="./assets/styles.css" />  
</head>  
<body>  
  <header class="topbar">
```

```

<div class="container">
  <div class="brand">
    <span class="logo">SIEM</span>
    <div>
      <h1>Alertas</h1>
      <p class="muted">Vista principal – listado, filtros y paginación</p>
    </div>
  </div>

  <div class="actions">
    <button id="btnRefresh" class="btn">Actualizar</button>
  </div>
</div>

</header>

<main class="container">
  <section class="card">
    <div class="card-header">
      <h2>Filtros</h2>
      <p class="muted">Se aplican contra <code>/alerts</code></p>
    </div>

    <form id="filtersForm" class="filters">
      <div class="field">
        <label for="status">Estado</label>
        <select id="status" name="status">
          <option value="">(todos)</option>
          <option value="open">open</option>
          <option value="ack">ack</option>
          <option value="closed">closed</option>
        </select>
      </div>

      <div class="field">
        <label for="group_key">group_key (host)</label>
        <input id="group_key" name="group_key" type="text" placeholder="ej: web-01" />
      </div>

      <div class="field">
        <label for="limit">Límite</label>
        <select id="limit" name="limit">
          <option value="25">25</option>
          <option value="50" selected>50</option>
          <option value="100">100</option>
          <option value="200">200</option>
        </select>
      </div>

      <div class="field buttons">
        <button class="btn primary" type="submit">Aplicar</button>
        <button id="btnClear" class="btn" type="button">Limpiar</button>
      </div>
    </form>
  </section>

  <section class="card">
    <div class="card-header row">
      <div>
        <h2>Listado</h2>
        <p id="resultMeta" class="muted"><--</p>
      </div>

      <div class="pager">
        <button id="btnPrev" class="btn" type="button"><- Prev</button>
        <button id="btnNext" class="btn" type="button">Next ></button>
      </div>
    </div>
  </section>
</main>

```

```

    </div>
</div>

<div id="errorBox" class="alert error hidden"></div>
<div id="infoBox" class="alert info hidden"></div>

<div class="table-wrap">
  <table class="table" aria-label="Tabla de alertas">
    <thead>
      <tr>
        <th>ID</th>
        <th>Estado</th>
        <th>Severidad</th>
        <th>group_key</th>
        <th>Título</th>
        <th>rule_id</th>
        <th>event_id</th>
        <th>created_at</th>
        <th></th>
      </tr>
    </thead>
    <tbody id="alertsTbody">
      <tr><td colspan="9" class="muted">Cargando...</td></tr>
    </tbody>
  </table>
</div>

<div class="card-footer">
  <small class="muted">
    Nota: "Severidad" no existe en <code>AlertOut</code>; se deja como placeholder ("–") para
    evitar inventar datos.
  </small>
</div>
</section>
</main>

<script src="./assets/app.js"></script>
<script src="./assets/alerts.js"></script>
</body>
</html>

```

4 Función general del archivo

El archivo `index.html` define la página principal del frontend.

Su objetivo es ofrecer una vista visual de las alertas generadas por el laboratorio.

Desde esta página el usuario puede:

- Ver el listado de alertas.
- Filtrar por estado.
- Filtrar por `group_key`.
- Elegir el límite de resultados.
- Pagar con Prev y Next.
- Refrescar manualmente el listado.
- Limpiar filtros.
- Acceder al detalle de una alerta.

Este archivo no contiene la lógica de conexión con la API. Esa parte está en:

```

frontend/assets/app.js
frontend/assets/alerts.js

```

La relación es:

```
index.html
  ↓
define estructura visual

styles.css
  ↓
aplica diseño

app.js
  ↓
define funciones comunes

alerts.js
  ↓
carga datos y actualiza la tabla
```

5 Estructura general del archivo

El archivo puede dividirse en seis bloques:

```
<!doctype html>
<html lang="es">
```

Declaración del documento HTML.

```
<head>
...
</head>
```

Metadatos, título y enlace al CSS.

```
<header class="topbar">
...
</header>
```

Cabecera superior de la página.

```
<main class="container">
...
</main>
```

Contenido principal.

Dentro de `main` hay dos secciones:

1. Filtros
2. Listado de alertas

Al final se cargan los scripts:

```
<script src="./assets/app.js"></script>
<script src="./assets/alerts.js"></script>
```

Visualmente:

```
index.html
├─ doctype
├─ html lang="es"
├─ head
│   ├─ charset
│   └─ viewport
```

```
| | title
| | styles.css
| body
| | header.topbar
| | | logo
| | | título
| | | botón Actualizar
| | main.container
| | | section.card filtros
| | | section.card listado
| | scripts
| | | app.js
| | | alerts.js
```

6 Análisis línea por línea

Declaración del tipo de documento

```
<!doctype html>
```

Indica al navegador que el documento usa HTML5.

Es la primera línea habitual en documentos HTML modernos.

Ayuda a que el navegador renderice la página en modo estándar.

Apertura del documento HTML

```
<html lang="es">
```

Abre el documento HTML.

El atributo:

```
lang="es"
```

indica que el idioma principal de la página es español.

Esto ayuda a:

- Accesibilidad.
- Lectores de pantalla.
- Motores de búsqueda.
- Correctores lingüísticos del navegador.

Apertura del head

```
<head>
```

Inicia la sección de metadatos del documento.

El contenido de `head` no se muestra directamente en la página, pero configura cómo se interpreta y presenta el documento.

Codificación de caracteres

```
<meta charset="utf-8" />
```

Define la codificación del documento como UTF-8.

Esto permite mostrar correctamente caracteres como:

```
á
é
ñ
—
...
```

Es importante porque la página contiene textos en español.

Configuración responsive

```
<meta name="viewport" content="width=device-width,initial-scale=1" />
```

Controla cómo se adapta la página a diferentes tamaños de pantalla.

Desglose:

```
width=device-width
```

Indica que el ancho de la página debe ajustarse al ancho real del dispositivo.

```
initial-scale=1
```

Indica que el zoom inicial será 1.

Esto es básico para que la página funcione correctamente en móviles, tablets y pantallas pequeñas.

Título de la pestaña

```
<title>SIEM Lab – Alertas</title>
```

Define el título que aparece en la pestaña del navegador.

No es el título visible dentro de la página.

Sirve para identificar la página en el navegador.

Enlace al CSS

```
<link rel="stylesheet" href="./assets/styles.css" />
```

Carga la hoja de estilos del frontend.

Desglose:

```
rel="stylesheet"
```

Indica que el recurso enlazado es una hoja de estilos CSS.

```
href="./assets/styles.css"
```

Ruta del archivo CSS.

Relación:

```
index.html
  ↓
assets/styles.css
```

Sin este archivo, la página seguiría funcionando, pero sin el diseño visual definido.

Cierre del `head`

```
</head>
```

Cierra la sección de metadatos.

Apertura del `body`

```
<body>
```

Inicia el contenido visible de la página.

Todo lo que ve el usuario está dentro de `body`.

Cabecera superior

```
<header class="topbar">
```

Define la cabecera superior de la página.

El elemento `header` se usa semánticamente para representar una zona de encabezado.

La clase:

```
class="topbar"
```

permite aplicar estilos desde `styles.css`.

En el CSS, `.topbar` define una barra superior fija visualmente, con fondo translúcido y borde inferior.

Contenedor interno de cabecera

```
<div class="container">
```

Crea un contenedor para limitar el ancho del contenido y centrarlo.

La clase `container` se reutiliza en varias partes del frontend.

En `styles.css`, `.container` define:

```
max-width: 1200px;
margin: 0 auto;
padding: 16px;
```

Esto evita que el contenido ocupe todo el ancho de pantallas grandes.

Bloque de marca

```
<div class="brand">
```

Agrupar el logo y los textos principales de la cabecera.

La clase `brand` se usa para alinear el logo y el texto horizontalmente.

Logo textual

```
<span class="logo">SIEM</span>
```

Muestra un logo simple con el texto `SIEM`.

Se usa `span` porque es un elemento en línea, aunque el CSS lo transforma visualmente en una caja.

La clase `logo` aplica el estilo visual del recuadro.

Contenedor de título

```
<div>
```

Agrupar el título `h1` y el subtítulo.

No tiene clase porque solo sirve como contenedor estructural.

Título principal visible

```
<h1>Alertas</h1>
```

Define el título principal de la página.

Indica que esta vista está centrada en alertas.

Subtítulo de la página

```
<p class="muted">Vista principal – listado, filtros y paginación</p>
```

Muestra una breve descripción de la vista.

La clase:

```
class="muted"
```

aplica un color más suave, usado para textos secundarios.

Cierre de contenedores de marca

```
</div>  
</div>
```

Cierra el contenedor del título y el bloque `.brand`.

Contenedor de acciones

```
<div class="actions">
```

Agrupar acciones de la cabecera.

En esta página contiene el botón de actualización.

Botón actualizar

```
<button id="btnRefresh" class="btn">Actualizar</button>
```

Define un botón para recargar manualmente el listado de alertas.

Desglose:

```
id="btnRefresh"
```

Identificador usado por JavaScript.

En `alerts.js`, se conecta así:

```
qs("btnRefresh").addEventListener("click", () => loadAlerts());
```

```
class="btn"
```

Aplica el estilo común de botón.

```
Actualizar
```

Texto visible del botón.

Cierre de cabecera

```
</div>
</div>
</header>
```

Cierra el contenedor de acciones, el contenedor principal de la cabecera y el elemento `header`.

Apertura del contenido principal

```
<main class="container">
```

Define el contenido principal de la página.

`main` es semánticamente el área central del documento.

La clase `container` mantiene el mismo ancho máximo que la cabecera.

Primera tarjeta: filtros

```
<section class="card">
```

Crea una sección visual en forma de tarjeta.

Esta primera tarjeta contiene los filtros.

La clase `card` aplica fondo, borde, sombra y redondeo.

Cabecera de la tarjeta de filtros

```
<div class="card-header">
```

Define la cabecera interna de la tarjeta.

Título de filtros

```
<h2>Filtros</h2>
```

Indica que esta sección contiene los controles de filtrado.

Explicación del endpoint usado

```
<p class="muted">Se aplican contra <code>/alerts</code></p>
```

Indica que los filtros se aplican contra el endpoint:

```
/alerts
```

El elemento:

```
<code>/alerts</code>
```

muestra el texto con estilo de código.

Punto importante: esto confirma que el frontend actual usa el endpoint básico `/alerts`, no `/alerts/ui`.

Cierre del header de filtros

```
</div>
```

Cierra `.card-header`.

Formulario de filtros

```
<form id="filtersForm" class="filters">
```

Define un formulario para aplicar filtros.

Desglose:

```
id="filtersForm"
```

Identificador usado por JavaScript.

En `alerts.js` se conecta así:

```
qs("filtersForm").addEventListener("submit", ...)
```

```
class="filters"
```

Aplica diseño en grid a los campos del formulario.

Campo de estado

```
<div class="field">
```

Agrupar una etiqueta y un control de formulario.

La clase `field` se usa para organizar visualmente cada filtro.

Etiqueta de estado

```
<label for="status">Estado</label>
```

Etiqueta asociada al selector de estado.

El atributo:

```
for="status"
```

conecta la etiqueta con el elemento que tiene:

```
id="status"
```

Esto mejora accesibilidad y usabilidad.

Selector de estado

```
<select id="status" name="status">
```

Crea un desplegable para filtrar por estado.

Desglose:

```
id="status"
```

Identificador usado por JavaScript.

```
name="status"
```

Nombre del campo dentro del formulario.

Opción todos

```
<option value="">( todos)</option>
```

Representa la opción sin filtro.

El valor vacío significa que no se enviará `status` al backend.

En `alerts.js`, si `state.status` está vacío, se envía como `null`:

```
status: state.status || null
```

Opciones de estado

```
<option value="open">open</option>
<option value="ack">ack</option>
<option value="closed">closed</option>
```

Permiten filtrar por los tres estados válidos de una alerta.

Estos estados coinciden con el backend:

```
AlertStatus = Literal["open", "ack", "closed"]
```

Cierre del campo estado

```
</select>
</div>
```

Cierra el selector y su contenedor.

Campo `group_key`

```
<div class="field">
  <label for="group_key">group_key (host)</label>
  <input id="group_key" name="group_key" type="text" placeholder="ej: web-01" />
</div>
```

Este bloque permite filtrar alertas por `group_key`.

Desglose del input:

```
id="group_key"
```

Identificador usado por JavaScript.

```
name="group_key"
```

Nombre del campo.

```
type="text"
```

Indica que es un campo de texto.

```
placeholder="ej: web-01"
```

Texto de ayuda que aparece cuando el campo está vacío.

Este filtro conecta con el backend:

```
GET /alerts?group_key=web-01
```

Campo límite

```
<div class="field">
  <label for="limit">Límite</label>
  <select id="limit" name="limit">
```

Permite seleccionar cuántas alertas se muestran por página.

Opciones de límite

```
<option value="25">25</option>
<option value="50" selected>50</option>
<option value="100">100</option>
<option value="200">200</option>
```

Define valores posibles para `limit`.

La opción:

```
selected
```

marca 50 como valor por defecto.

Esto coincide con el estado inicial en `alerts.js`:

```
limit: 50
```

Botones del formulario

```
<div class="field buttons">
```

Agrupar los botones de acción del formulario.

Las clases:

```
field buttons
```

permiten aplicar estilo específico.

Botón aplicar

```
<button class="btn primary" type="submit">Aplicar</button>
```

Envía el formulario.

Desglose:

```
class="btn primary"
```

Aplica estilo de botón principal.

```
type="submit"
```

Hace que el botón dispare el evento `submit` del formulario.

En `alerts.js`, el submit:

- evita recargar la página
- lee filtros
- pone offset a 0
- actualiza URL
- recarga alertas

Botón limpiar

```
<button id="btnClear" class="btn" type="button">Limpiar</button>
```

Limpia filtros.

Desglose:

```
id="btnClear"
```

Identificador usado por JavaScript.

```
type="button"
```

Evita que el botón envíe el formulario.

En `alerts.js`, este botón limpia `status`, `group_key` y `offset`.

Cierre de filtros

```
</div>  
</form>  
</section>
```

Cierra el bloque de botones, el formulario y la tarjeta de filtros.

Segunda tarjeta: listado

```
<section class="card">
```

Crea la tarjeta que contiene el listado de alertas.

Cabecera del listado

```
<div class="card-header row">
```

Cabecera de la tarjeta.

La clase `row` permite colocar elementos en horizontal: título a la izquierda y paginación a la derecha.

Bloque del título del listado

```
<div>
  <h2>Listado</h2>
  <p id="resultMeta" class="muted"></p>
</div>
```

Contiene el título y una línea de metadatos.

El elemento:

```
<p id="resultMeta" class="muted"></p>
```

se actualiza desde JavaScript.

En `alerts.js`:

```
qs("resultMeta").textContent =
  `limit=${state.limit} · offset=${state.offset} · resultados=${data.length}`;
```

Esto permite mostrar información del listado actual.

Contenedor de paginación

```
<div class="pager">
```

Agrupar los botones de paginación.

Botón anterior

```
<button id="btnPrev" class="btn" type="button">← Prev</button>
```

Permite volver a la página anterior.

En `alerts.js`, resta `limit` al `offset`.

Botón siguiente

```
<button id="btnNext" class="btn" type="button">Next →</button>
```

Permite avanzar a la siguiente página.

En `alerts.js`, suma `limit` al `offset`.

Cierre de cabecera del listado

```
</div>
</div>
```

Cierra `.pager` y `.card-header`.

Caja de error

```
<div id="errorBox" class="alert error hidden"></div>
```

Elemento usado para mostrar errores.

Desglose:

```
id="errorBox"
```

Identificador usado por JavaScript.

```
class="alert error hidden"
```

Clases de estilo.

`hidden` hace que inicialmente no se muestre.

En `app.js`, las funciones `show()` y `hide()` muestran u ocultan este elemento.

Caja de información

```
<div id="infoBox" class="alert info hidden"></div>
```

Elemento usado para mostrar mensajes informativos.

Por ejemplo, cuando no hay más resultados en una página.

Contenedor de tabla

```
<div class="table-wrap">
```

Envuelve la tabla.

La clase `table-wrap` permite gestionar desbordamiento horizontal.

Esto es útil si la tabla es más ancha que la pantalla.

Tabla de alertas

```
<table class="table" aria-label="Tabla de alertas">
```

Crea la tabla.

Desglose:

```
class="table"
```

Aplica estilos de tabla.

```
aria-label="Tabla de alertas"
```

Mejora accesibilidad describiendo la tabla para tecnologías asistivas.

Cabecera de tabla

```
<thead>
  <tr>
    <th>ID</th>
    <th>Estado</th>
    <th>Severidad</th>
    <th>group_key</th>
    <th>Título</th>
    <th>rule_id</th>
    <th>event_id</th>
    <th>created_at</th>
    <th></th>
  </tr>
</thead>
```

Define las columnas de la tabla.

Columnas:

```
ID
Estado
Severidad
group_key
Título
rule_id
event_id
created_at
acción
```

La última columna está vacía porque se usa para el botón/enlace `Ver` .

Cuerpo de tabla dinámico

```
<tbody id="alertsTbody">
  <tr><td colspan="9" class="muted">Cargando...</td></tr>
</tbody>
```

Este bloque es clave.

El `tbody` tiene:

```
id="alertsTbody"
```

JavaScript lo usa para insertar filas dinámicamente.

Inicialmente muestra:

```
Cargando...
```

El atributo:

```
colspan="9"
```

hace que la celda ocupe las 9 columnas de la tabla.

Cuando `alerts.js` recibe datos, reemplaza este contenido.

Cierre de tabla

```
</table>
</div>
```

Cierra la tabla y su contenedor.

Pie de tarjeta

```
<div class="card-footer">
```

Define un pie informativo dentro de la tarjeta.

Nota sobre severidad

```
<small class="muted">
  Nota: "Severidad" no existe en <code>Alert0ut</code>; se deja como placeholder ("–") para evitar
  inventar datos.
</small>
```

Esta nota es importante técnicamente.

Explica que el endpoint usado actualmente devuelve `Alert0ut`, que no incluye severidad.

Por tanto, la columna `Severidad` se deja como:

```
–
```

Esto evita mostrar información falsa.

Para mostrar severidad real habría que consumir:

```
GET /alerts/ui
```

porque `AlertUI0ut` sí incluye:

```
event_severity
```

Cierre del contenido principal

```
</section>
</main>
```

Cierra la tarjeta de listado y el bloque principal de la página.

Carga de `app.js`

```
<script src="./assets/app.js"></script>
```

Carga el archivo JavaScript común.

Este archivo define funciones usadas por `alerts.js`, como:

```
qs()
show()
```

```
hide()
fmtDate()
statusBadgeClass()
apiFetch()
getQueryParam()
setQueryParams()
```

Debe cargarse antes que `alerts.js`, porque `alerts.js` depende de estas funciones.

Carga de `alerts.js`

```
<script src="./assets/alerts.js"></script>
```

Carga la lógica específica de la página principal.

Este archivo:

- Lee filtros.
- Llama a la API.
- Renderiza filas.
- Gestiona paginación.
- Gestiona botones.

Cierre del documento

```
</body>
</html>
```

Cierra el cuerpo y el documento HTML.

7 Relación con el flujo técnico del laboratorio

`index.html` representa la vista principal de alertas.

La relación con el backend es:

```
index.html
  ↓
carga app.js y alerts.js
  ↓
alerts.js ejecuta loadAlerts()
  ↓
apiFetch("/alerts")
  ↓
FastAPI GET /alerts
  ↓
PostgreSQL tabla alerts
  ↓
respuesta JSON
  ↓
renderRows()
  ↓
tabla HTML
```

Flujo visual:

```
Usuario abre index.html
  ↓
Se muestra la estructura HTML
  ↓
```

```
JavaScript carga alertas
↓
La tabla se rellena dinámicamente
↓
Usuario filtra o pagina
↓
Se vuelve a consultar /alerts
```

8 Errores típicos o puntos importantes

`index.html` no contiene los datos

La tabla no tiene alertas escritas manualmente.

Solo tiene un placeholder:

```
Cargando...
```

Los datos reales llegan desde JavaScript.

El frontend usa `/alerts`, no `/alerts/ui`

Esto limita los datos disponibles.

Por eso la severidad aparece como `-`.

Para mostrar más contexto habría que cambiar la lógica de `alerts.js`.

Los IDs son fundamentales

Elementos como:

```
btnRefresh
filtersForm
status
group_key
limit
btnClear
btnPrev
btnNext
alertsTbody
errorBox
infoBox
resultMeta
```

son usados directamente por JavaScript.

Si se cambia un `id` en HTML, habría que cambiar también el JavaScript.

El orden de los scripts importa

Primero se carga:

```
<script src="./assets/app.js"></script>
```

Después:

```
<script src="./assets/alerts.js"></script>
```

Si se invierte el orden, `alerts.js` podría fallar porque usa funciones definidas en `app.js`.

El botón Limpiar usa type="button"

Esto evita que el botón envíe el formulario.

Si fuera submit, podría comportarse como el botón Aplicar.

La tabla usa colspan="9"

Como la tabla tiene 9 columnas, el mensaje inicial ocupa toda la fila.

Si se añaden o quitan columnas, conviene ajustar este número.

9 Comandos útiles relacionados

Servir el frontend:

```
cd ~/siem-lab/frontend
python3 -m http.server 5173
```

Abrir la página principal:

```
http://localhost:5173/index.html
```

Probar el endpoint que usa esta página:

```
curl http://localhost:8000/alerts
```

Probar filtros equivalentes:

```
curl "http://localhost:8000/alerts?limit=50&offset=0&status=open"
```

Probar filtro por group_key :

```
curl "http://localhost:8000/alerts?group_key=server-01"
```

Abrir Swagger:

```
http://localhost:8000/docs
```

Abrir una alerta concreta desde la interfaz:

```
http://localhost:5173/alert.html?id=1
```